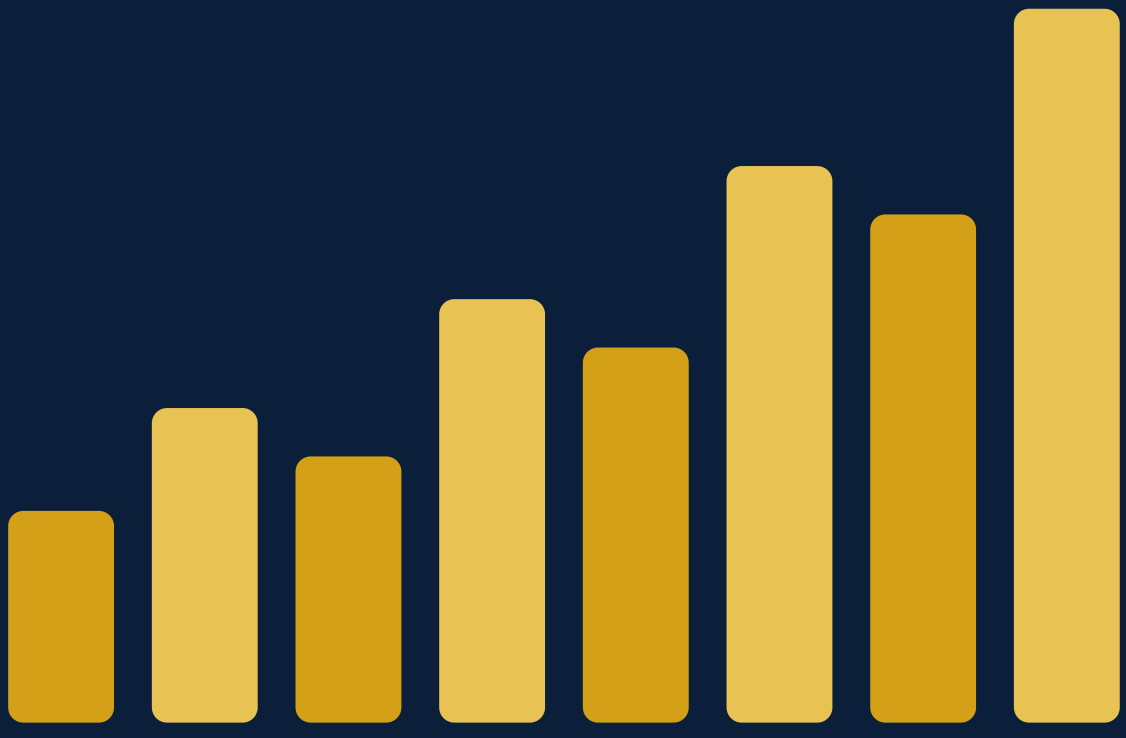
PROJECT REPORT

Gold Price Prediction

Using a Random Forest Regressor and financial market data



Report date: October 11, 2026

SECTION 1

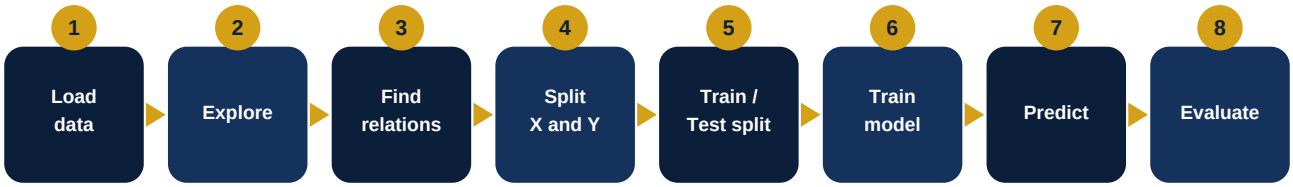
Project Overview

This project predicts the **price of gold** using a machine learning model. It uses a dataset of historical prices of several financial assets and learns how those assets are related to the gold price.

Think of it like this: you want to guess today's gold price. You look at other things such as the stock market, oil prices, silver prices and currency rates. The computer learns from past data how these things affect gold, and then predicts gold prices for new data.

2290	6	4	0.9887
rows of data	columns	input features	R^2 score

THE WORKFLOW AT A GLANCE



SECTION 2

The Dataset

The dataset is a CSV file (like a spreadsheet) with the following columns:

Column	Meaning	Role
Date	The date of the record	Dropped (just a label)
SPX	S&P 500 index (US stock market)	Input feature
GLD	Gold price	**Target to predict**
USO	Oil price	Input feature
SLV	Silver price	Input feature
EUR/USD	Euro to US Dollar exchange rate	Input feature

SECTION 3

Step-by-Step Explanation

1 Importing the Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn import metrics
```

In simple words

numpy and **pandas** are tools to work with numbers and tables (like Excel in Python). **matplotlib** and **seaborn** draw graphs and charts. **sklearn** is a machine learning library: it splits data into training and testing parts, provides the **Random Forest Regressor** model, and checks how good the predictions are.

2 Loading the Data

```
gold_data = pd.read_csv('/content/gold price dataset.csv')
```

In simple words

We load a CSV file (like a spreadsheet) that contains the gold price data.

3 Looking at the Data

```
gold_data.head()           # First 5 rows
gold_data.tail()           # Last 5 rows
gold_data.shape            # (2290, 6) -> 2290 rows, 6 columns
gold_data.info()           # Column types and missing values
gold_data.isnull().sum()   # Counts missing values
gold_data.describe()       # Statistics (mean, min, max, etc.)
```

In simple words

We explore the data to understand what it looks like, how many rows and columns it has, whether any values are missing (**answer: no missing values**), and basic statistics such as the average, minimum and maximum prices.

4 Finding Relationships (Correlation)

```
correlation = gold_data.corr()
sns.heatmap(correlation, annot=True, cmap='Blues')
print(correlation['GLD'])
```

In simple words
Correlation tells us how two things are related. A **positive** correlation means when one goes up, the other also goes up. A **negative** correlation means when one goes up, the other goes down. The heatmap is just a colorful picture of these relationships.

Asset vs. Gold	Result	What it means
SLV (Silver)	**0.87**	Very high positive: when silver rises, gold usually rises too
USO (Oil)	**-0.19**	Slight negative: when oil rises, gold tends to dip a little
SPX and **EUR/USD**	Very weak	Almost no linear relationship with gold

5 Checking the Distribution of Gold Price

```
sns.distplot(gold_data['GLD'], color='green')
```

In simple words
This draws a curve showing how gold prices are spread out. Most gold prices are between roughly **110 and 130**.

6 Splitting Features and Target

```
X = gold_data.drop(['Date','GLD'], axis=1)
Y = gold_data['GLD']
```

In simple words
X (features) is the input data used to predict: SPX, USO, SLV and EUR/USD. **Y (target)** is the answer we want to predict: GLD, the gold price. The **Date** column is removed because it is only a label and is not useful for prediction.

7 Splitting into Training and Testing Data

```
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=2)
```

In simple words

80% of the data is used to train the model (teach it) and **20%** is used to test it (check if it learned well). `random_state=2` makes sure we get the same split every time we run the code.

8 Training the Model (Random Forest Regressor)

```
regressor = RandomForestRegressor(n_estimators=100)
regressor.fit(X_train, Y_train)
```

In simple words

Imagine you ask **100 different people** to guess the gold price. Each person looks at different parts of the data. Then you take the **average of all 100 guesses**, which is usually very accurate. That is what Random Forest does: it builds 100 decision trees and averages their results. `n_estimators=100` means we use 100 trees.

9 Making Predictions

```
test_data_prediction = regressor.predict(X_test)
```

In simple words

The model looks at the test data (the 20% it has never seen) and predicts gold prices for it.

10 Checking How Good the Model Is

```
error_score = metrics.r2_score(Y_test, test_data_prediction)
print("R squared error : ", error_score)
```

In simple words

The **R^2 score** tells us how well the model predicts: 1.0 is a perfect prediction and 0.0 is a very bad one. Our score is **0.9887**, which is very close to 1. This means the model explains about **98.87%** of the variation in gold prices on the test data, which is excellent.

11 Comparing Actual vs Predicted Values

```
Y_test = list(Y_test)

plt.plot(Y_test, color='blue', label='Actual Value')
plt.plot(test_data_prediction, color='green', label='Predicted Value')
plt.title('Actual Price vs Predicted Price')
plt.xlabel('Number of values')
plt.ylabel('GLD Price')
plt.legend()
plt.show()
```

In simple words

We draw a graph with two lines: a **blue line** for real gold prices and a **green line** for predicted gold prices. If the two lines are very close, the model is doing a great job. In our graph they overlap almost perfectly, which confirms the model is very accurate.

SECTION 4

Results and Summary

Step	What We Did
1	Loaded the gold price dataset
2	Explored the data (rows, columns, missing values, stats)
3	Found relationships between gold and other assets
4	Split data into inputs (X) and output (Y)
5	Split data into training (80%) and testing (20%)
6	Trained a Random Forest model with 100 trees
7	Predicted gold prices for the test data
8	Checked performance: R^2 score of 0.9887 (about 98.87%)
9	Drew a graph to compare actual vs predicted prices

FINAL RESULT

The model can predict gold prices very accurately using other financial data such as silver, oil, the stock market and currency rates.

KEY TAKEAWAY

This project shows how machine learning can be used to predict gold prices. It uses a **Random Forest Regressor**, a powerful algorithm that combines many decision trees to make accurate predictions. The model reached an **R^2 score of 0.9887** (about 98.87%), which means it is very reliable on this dataset.